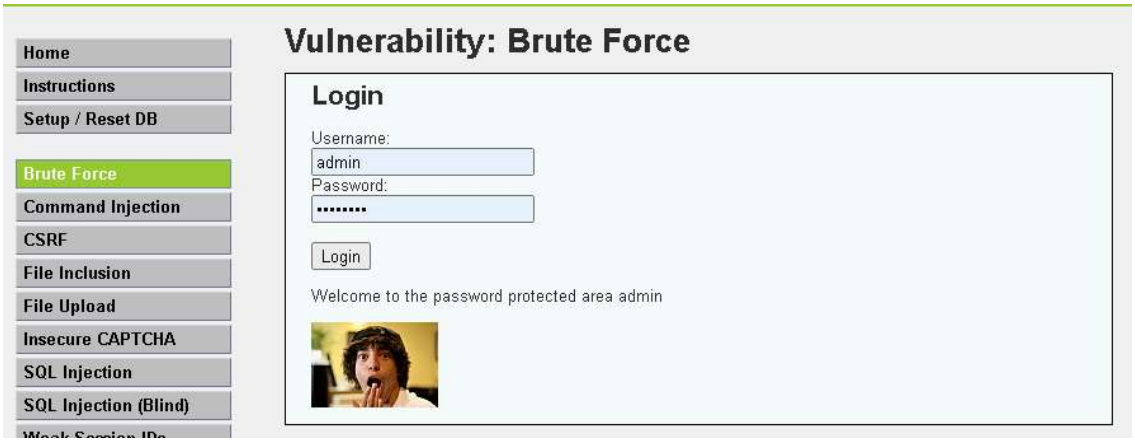
CyberSecurity-Mini Project

Part A - DVWA

1. **There are several vulnerabilities/threats in the DVWA. Please identify at-least 8 vulnerabilities/threats in the application.**

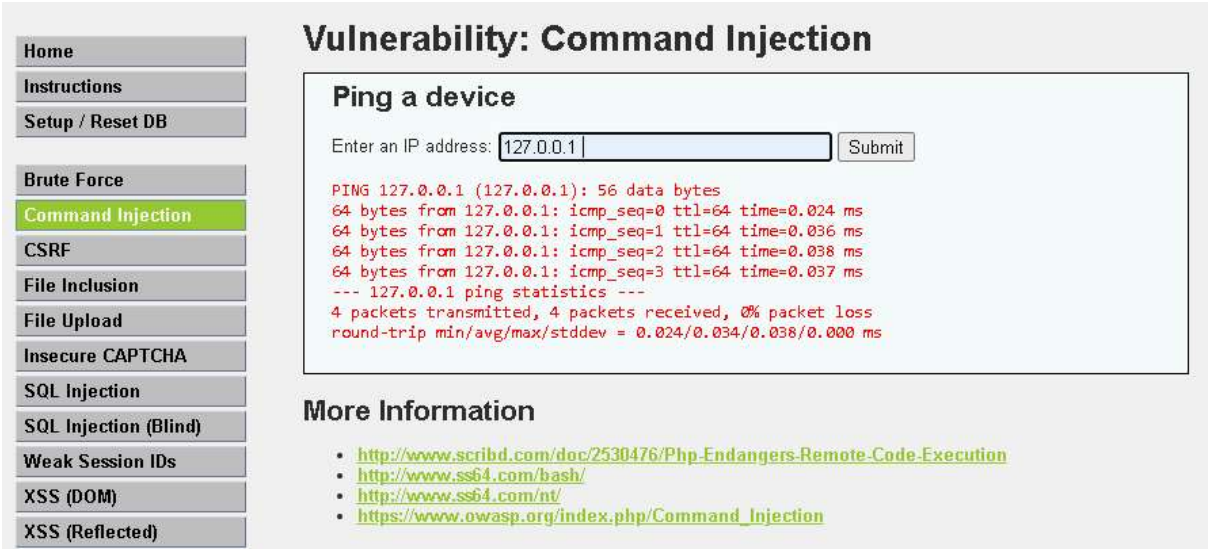
Vulnerability1 : Brute Force for admin password

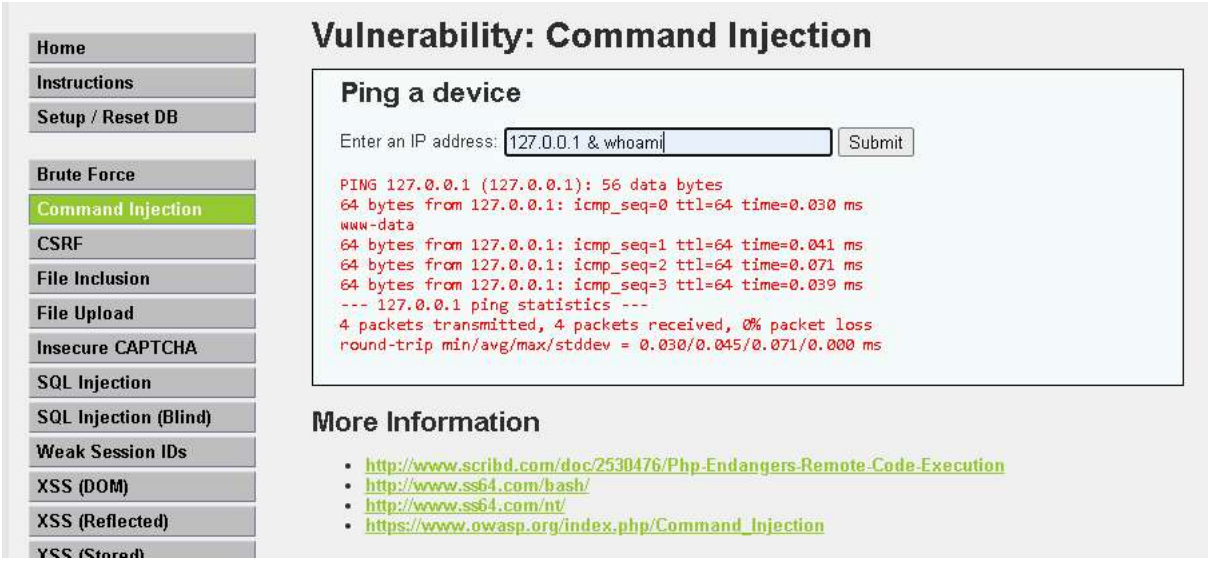


Method : Rainbow table -> try it because no rate limit

Username : “admin” , Password : “password”

Vulnerability2 : Code Injection to get host server information





Method:use “&” to run multiple commands on one line

I think background is common “ ping [input]”, no input validation so easy to inject

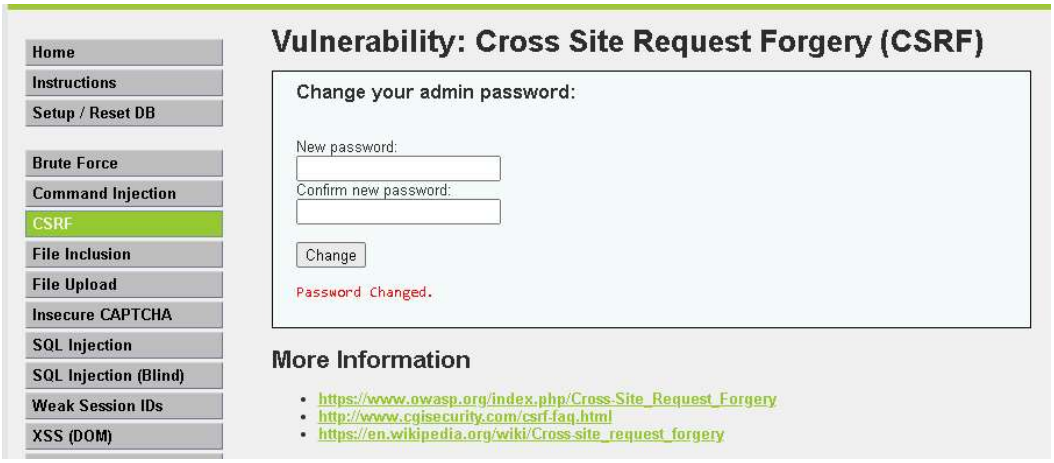
Vulnerability3 : Cross Site Request Forgery (CSRF) to change user password

ไม่ได้กรอกอะไรเลย แต่ใช้ url ก็สามารถเปลี่ยนรหัสได้แล้ว

url :

“http://localhost/vulnerabilities/csrf/?password_new=[new_password]&password_conf=[new_password]&Change=Change#”

Description : ไม่ต้องทำผ่าน webpage ก็ได้ แค่ให้เข้าของกดลิ้งนี้ ก็สามารถเปลี่ยนรหัสอีกฝ่ายได้แล้ว

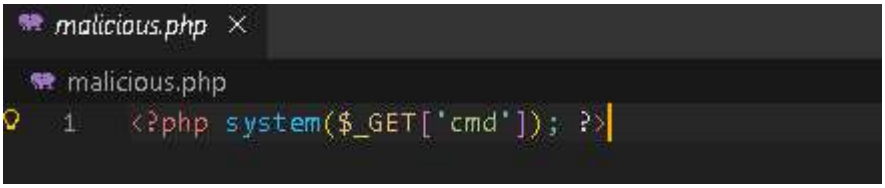


password เดิมใช้เข้าไม่ได้แล้ว

Vulnerability4 : Execute command through url and file we upload

Description : upload file และสามารถรันไฟล์ใน server ได้ เช่น

file ที่ใช้

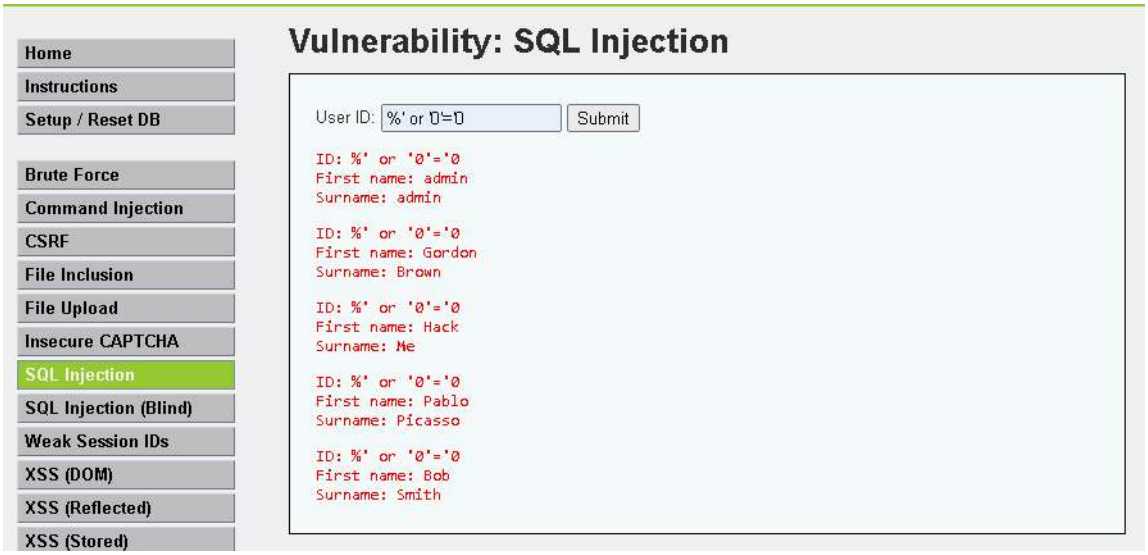


เมื่อ upload ไฟล์รันอยู่ที่ server เราสามารถรัน command โดยส่ง command ผ่าน url เช่น

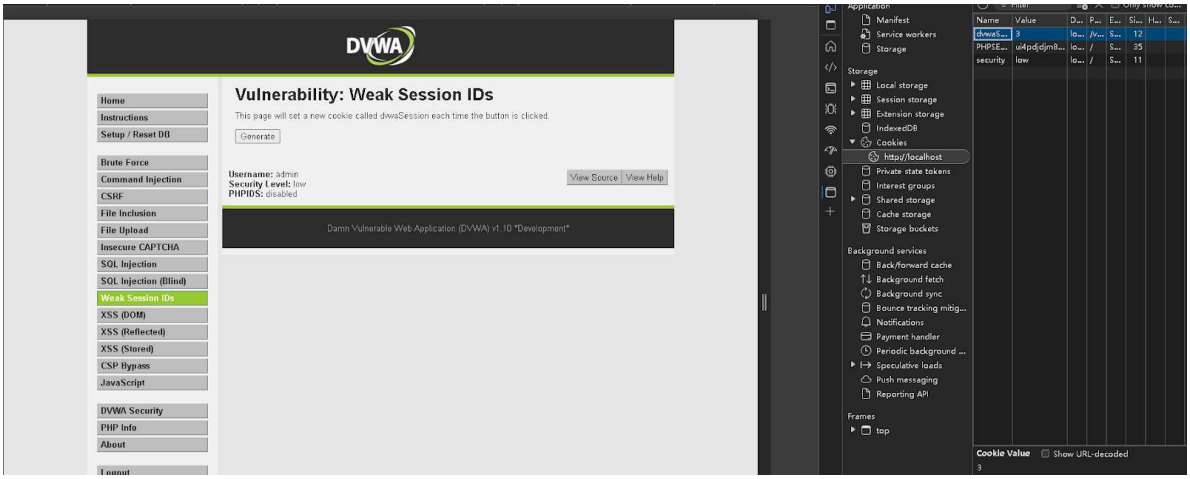
ได้ whoami มา

Vulnerability5 : SQL Injection ในการดึงข้อมูลจาก Database มาโชว์ (ได้ของทุก user)

Description : เดาว่า background คงเป็น WHERE userId = '{user_input}' เราสามารถเพิ่มเงื่อนไขได้ โดย input = %’ or ‘0’ = ‘0 ทำให้รวมแล้วได้ WHERE userId = ‘%’ or ‘0’ = ‘0’ ซึ่งเงื่อนไขหลังถูกเสมอ ทำให้ได้ทุกอย่างที่ select ออกมา



Vulnerability6 : Weak Session IDs

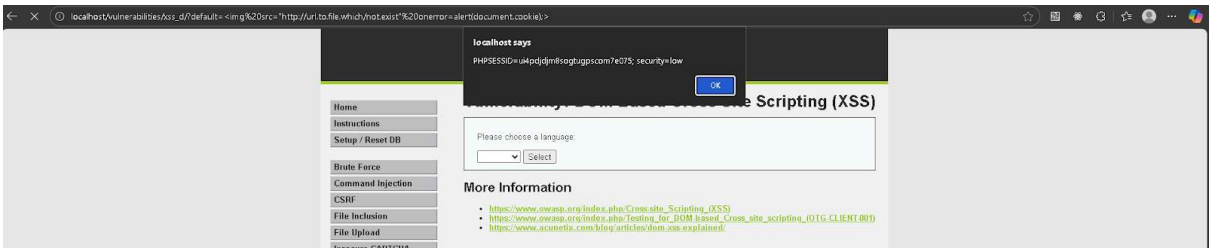


พบว่าเมื่อ กด generate ค่าจะเพิ่มขึ้น 1 (1->2->3), easy to guess others session cookie -> ปลอมตัวเป็นคนอื่นได้

Vulnerability7 : XSS (DOM)

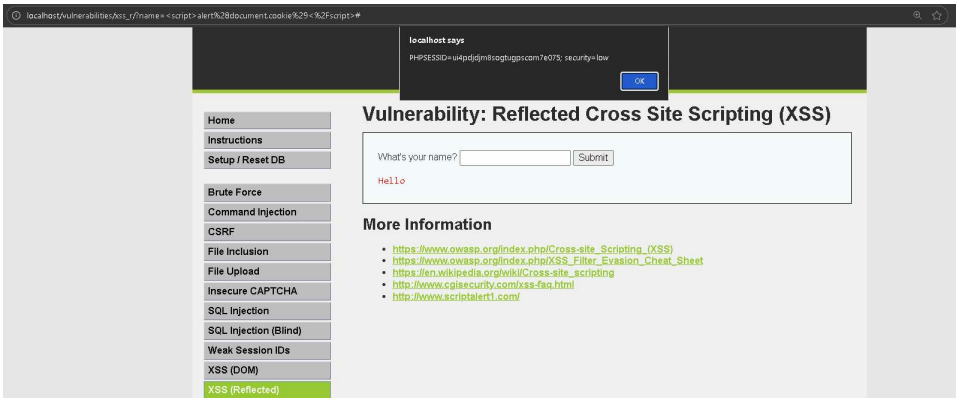
เขียนโค้ดเพิ่มในลง url ทำให้สามารถดึงข้อมูล/แก้ html ได้

Method : ดู url ว่าแก้อะไรไปบ้าง เราเพิ่มตัวเลือก click me (แก้ html)ในรูปแรก และ alert ให้โชว์ document.cookie (ดึงข้อมูล) ในรูปสอง



Vulnerability8 : XSS (Reflected)

Description : เมื่อ response มีการใช้ค่าที่รับจาก user ทำให้เราสามารถเขียน script นี้ได้, ดึงข้อมูลอย่าง cookie คล้ายๆ อันก่อนหน้าแต่เปลี่ยนที่ใส่ script ไปในช่อง input แล้วเมื่อได้ผลกลับมา มันจะสามารถดึงข้อมูลหรือโชว์ข้อมูลได้ ในที่นี้ input = <script> alert(document.cookie)</script> (โชว์ข้อมูล)

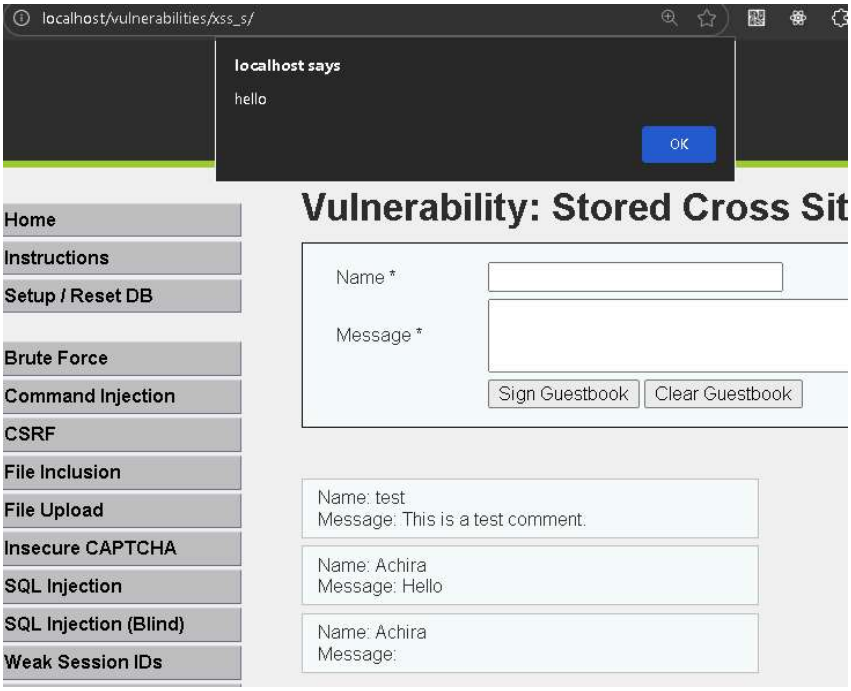


Vulnerability9 : XSS (Stored)

Description : คล้ายๆ กับ Reflected แต่ reflected เกิดจาก input เรา แต่อันนี้เกิดจาก ค่าจาก DB เราส่ง input นี้เข้าไปใน DB เมื่อคนอื่นดึงค่านี้มาโชว์ ก็จะโดน

Summary : ค่านี้เก็บใน database -> stored ถ้ามีคนเปิดหน้านี้จะโดน Script นี้หมด

ภาพแรก input = <script>alert(“hello”)</script>



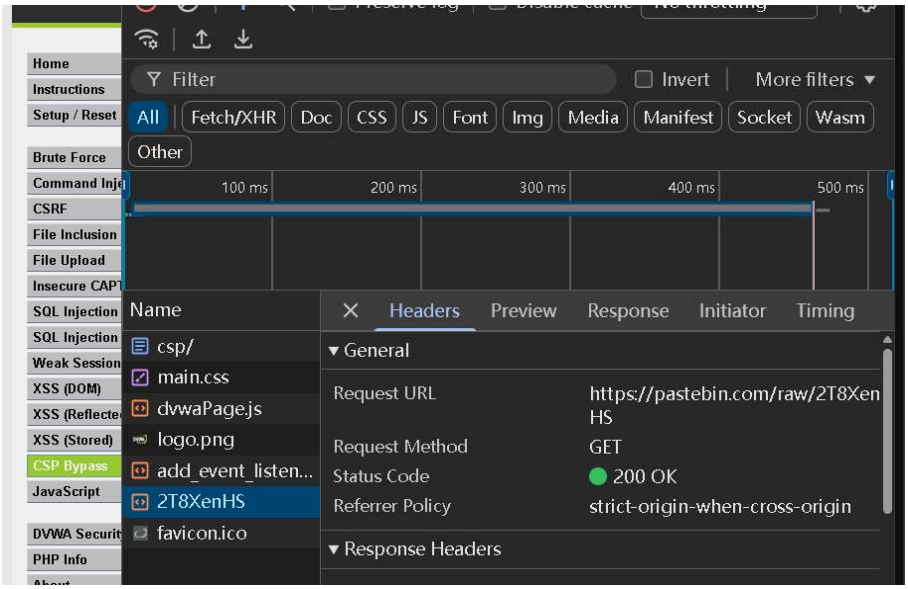
หรือ สามารถ : redirect ไปยังหน้าที่เราต้องการ

input = <script>location="https://youtu.be/dQw4w9WgXcQ"</script>

Vulnerability10 : CSP Bypass

CSP : 

Description : CSP ช่วยจำกัดได้ว่าให้โหลดของมาจากไหนได้บ้างเหมือน White list ที่ web นี้เชื่อใจ แต่ Bypass = แหล่งเชื่อใจไม่ปลอดภัยเช่น pastebin เราสามารถเอา script ไปแปะได้



Vulnerability11 : JavaScript

Description : JavaScript ที่ใช้ยืนยัน/ควบคุม (เช่น token generation) อยู่ใน frontend ถูกส่งไปยังผู้ใช้ (client-side) และผู้ใช้สามารถเปิดดู/แก้ไขได้ เพราะเป็น client-side logic ทำให้ ถ้าระบบพึ่งการตรวจสอบหรือควบคุมเฉพาะบนฝั่งclient ⟶ ผู้โจมตีสามารถ ดูโค้ด, ทำความเข้าใจ, แก้ไข payload เพื่อข้ามการตรวจสอบนั้นได้.

logic ที่เห็น

```
function generate_token() {
    var phrase = document.getElementById("phrase").value;
    document.getElementById("token").value = md5(rot13(phrase));
}

generate_token();
```

แก้ token ให้เป็น token ที่มีค่าเท่ากับ success

```
>document.getElementById("token").value = md5(rot13("success"))
<'38581812b435834ebf84ebcc2c6424d6'
```

Vulnerability12 : File Inclusion

Description : เมื่อ url มีการเปิดดูไฟล์อื่นๆ ภายใน server เราก็แค่ไปเปลี่ยน url ก็สามารถเปิด ไฟล์อื่นๆ ได้แล้ว เพราะไม่มีการinput validation check อย่าง ../ escape -> ได้ข้อมูล

url : localhost/vulnerabilities/fi/?page=../../hackable/flags/fi.php

2. **For each vulnerability, calculate a risk score based on 4 criterias of OWASP (Exploitability, Weakness Prevalence, Weakness Detectability, and Technical Impacts.)**

Threat Agents	Exploitability	Weakness Prevalence	Weakness Detectability	Technical Impacts	Business Impacts
Appli-cation Specific	Easy: 3	Widespread: 3	Easy: 3	Severe: 3	Business Specific
	Average: 2	Common: 2	Average: 2	Moderate: 2	
	Difficult: 1	Uncommon: 1	Difficult: 1	Minor: 1	

Risk Score คำนวณโดย ((E + WP + (4 - WD) + T)/12)*10 เพราะ Weakness Detectability เจอยาก ควร higher risk แต่เพื่อให้มี 10 เต็ม เลยทำให้เป็น 4- WD

Vulnerability1 : Brute Force for admin password

Exploitability : 3 : Easy, basically just try a password

Weakness prevalence : 3 : Weak + reused password still common and some app no rate-limit

Weakness detectability : 3 : Login fail should are detectable in logs

Technical impact : 3 : Account takeover -> severe

Risk Score : ((3+3+(4 - 3)+3) /12) * 10 = 8.33

Vulnerability2 : Code Injection to get host server information

Exploitability : 2 : It's not that simple — you still need to understand the app structure to extract high-quality information.

Weakness prevalence : 2 : No that common compared to SQLi and XSS

Weakness detectability : 1 : Hard because no log for input

Technical impact : 3 : Full command execution

Risk Score : ((2+2+(4 - 1)+3) /12) * 10 = 8.33

Vulnerability3 : Cross Site Request Forgery (CSRF) to change user password

Exploitability : 3 : Easy, because URL is not that hard to understand

Weakness prevalence : 2 : Most application can handle well

Weakness detectability : 1 : Hard, it looks like user change by themself

Technical impact : 3 : Change password -> Account takeover -> severe

Risk Score : ((3+2+(4 - 1)+3) /12) * 10 = 9.17

Vulnerability4 : Execute command through url and file we upload

Exploitability : 2 : It's not that simple — you still need to understand the app structure to extract high-quality information.

Weakness prevalence : 1 : Not that common, most have handle this problem

Weakness detectability : 1 : Hard, if file already get into the server

Technical impact : 3 : Remote code execution on host. Complete server takeover.

Risk Score : ((2+1+(4 - 1)+3) /12) * 10 = 7.5

Vulnerability5 : SQL Injection ในการดึงข้อมูลจาก Database มาโชว์ (ได้ของทุก user)

Exploitability : 3 : Easy, just understand SQL is enough

Weakness prevalence : 3 : Common in dynamic query app

Weakness detectability : 1 : Hard because it looks like normal data fetch

Technical impact : 3 : Get all data save in same database

Risk Score : ((3+3+(4 - 1)+3) /12) * 10 = 10

Vulnerability6 : Weak Session IDs

Exploitability : 3 : Easy because cookie is very predictable

Weakness prevalence : 3 : Common in aged system, bad algorithm for cookie

Weakness detectability : 3 : Easy detect from log -> repeated use

Technical impact : 3 : full account takeover

Risk Score : ((3+3+(4 - 3)+3) /12) * 10 = 8.33

Vulnerability7 : XSS (DOM)

Exploitability : 3 : Easy, just modify url with JS knowledge

Weakness prevalence : 3 : common in web that manipulate DOM without sanitization

Weakness detectability : 1 : Hard to detect server-side because payload executes in client

Technical impact : 3 : Can steal session tokens, perform actions as user, load malware

Risk Score : ((3+3+(4 - 1)+3) /12) * 10 = 10.00

Vulnerability8 : XSS (Reflected)

Exploitability : 3 : Easy, just create fake link -> convince target to open

Weakness prevalence : 3 : common in web that reflect unsanitized input into responses

Weakness detectability : 2 : detectable but not easy because payload appear in server

Technical impact : 3 : Can steal session tokens, Inject UI overlay to phish credentials, Redirect to malicious site

Risk Score : ((3+3+(4 - 2)+3) /12) * 10 = 9.17

Vulnerability9 : XSS (Stored)

Exploitability : 3 : Easy, just if found where to upload

Weakness prevalence : 2 : common in web that poorly sanitized input sinks

Weakness detectability : 2 : detectable but not easy because payload appear in server

Technical impact : 3 :Delivered to many users without repeated interaction from attacker

Risk Score : ((3+2+(4 - 2)+3) /12) * 10 = 8.33

Vulnerability10 : CSP Bypass

Exploitability : 2 : Not easy because require knowledge of page, script and find bypass

Weakness prevalence : 2 : misconfigurations and overly-permissive policies are still common

Weakness detectability : 1 : Hard to detect server-side since bypasses occur in the browser

Technical impact : 3 : Script execution, enabling XSS, token theft

Risk Score : ((3+2+(4 - 2)+3) /12) * 10 = 8.33

Vulnerability11 : JavaScript

Exploitability : 3 : Easy , just can read and reverse engineer JS

Weakness prevalence : 3 : Common mistake that dev put token generation logic in frontend

Weakness detectability : 1 : Hard to detect because attacker uses valid-looking tokens

Technical impact : 3 : grant unauthorized access or privileges; can lead to data exposure

Risk Score : ((3+3+(4 - 1)+3) /12) * 10 = 10

Vulnerability12 : File Inclusion

Exploitability : 2 : requires knowledge about application structure -> path

Weakness prevalence : 2 : Present in apps that build include paths in url

Weakness detectability : 1 : Hard to detect because normal requests can look legitimate.

Technical impact : 3 : Sensitive file disclosure, local file inclusion ⟶ information leak

Risk Score : ((2+2+(4 - 1)+3) /12) * 10 = 8.33

3. **For each vulnerability, suggest/show a fix for it. If it is a threat (cannot be fixed), please suggest a mitigation methodology. Please highlight/explain the concept clearly.**

Vulnerability1 : Brute force

Fix:

1. Rate limit — block guessing.
2. MFA — Multi-factor Authentication.
3. Strong password policy — avoid weak passwords.
4. Hash high-cost algorithm — raise brute force computed cost.

Vulnerability2 : Code injection

Fix:

1. Remove input execution — separate executable code from data.
2. Input validation — only allowed values.
3. Sandboxing/Isolation — Execute in an isolated env

Vulnerability3 : CSRF (change password)

Fix:

1. SameSite cookies / check Origin.
2. Require re-auth or MFA for sensitive operation.

Vulnerability4 : Execute command via uploaded file

Fix:

1. Disable file execution — make uploaded file unexecutable
2. Validate file : Scan uploads
3. Sandboxing/Isolation — Execute in an isolated env

Vulnerability5 : SQL Injection (read all users)

Fix:

1. Input validation — sanitized input
2. Never pass raw user input directly into a query call
3. Input as data, not executable code

Vulnerability6 : Weak Session IDs

Fix:

1. Use unpredictable Session IDs
2. Rotate IDs on privilege change and short TTL.
3. Detects token reuse and revokes compromised sessions.

Vulnerability7 : XSS (DOM)

Fix:

1. Use safe textContent which will treat input as plain text only.
2. Context-aware output encoding before insertion. -> it never becomes code
3. Auto-escaping libraries (React) -> sanitize user data before reaches the DOM

Vulnerability8 : XSS (Reflected)

Fix:

1. Context-aware escaping for HTML/JS/URLs. Output data stays data, never becomes code.
2. Input validation & length limits. -> block known bad string early, reduce attack surface
3. WAF + scanner rules to detect and block known attack payload.

Vulnerability9 : XSS (Stored)

Fix:

1. Input validation & length limits. -> block known bad string early, reduce attack surface
2. WAF + scanner rules to detect and block known attack payload.
3. Periodically re-scan and sanitize legacy content.

Vulnerability10 : CSP Bypass

Fix:

1. Choose a trustable and secure domain for CSP
2. Strengthen CSP

Vulnerability11 : Client-side token generation / JS-exposed logic

Fix:

1. Move auth/token_gen logic to server-side.
2. Try to put JS in server-side not client-side
3. Log most operation

Vulnerability12 : File Inclusion

Fix:

1. Use whitelist mapping id⟶filename, never include raw user path.
2. Permission restriction -> least privilege, not allow to access
3. Hide Path, dont lets user know insided application structure
4. Monitor file-access logs and isolate app runtime.